

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

**ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ № 6
«ВВЕДЕНИЕ В СУБД MONGODB. УСТАНОВКА MONGODB.
НАЧАЛО РАБОТЫ С БД» по дисциплине «Проектирование и
реализация баз данных»**

Обучающийся Иванова Аайа Гаврильевна
Факультет прикладной информатики
Группа К3241
Направление подготовки 09.03.03 Прикладная информатика
Образовательная программа Мобильные и сетевые технологии
Преподаватель Говорова Марина Михайловна, Белов Александр Олегович

Санкт-Петербург
2025/2026

Цель: овладеть практическими навыками работы с CRUD-операциями, с вложенными объектами в коллекции базы данных MongoDB, агрегации и изменения данных, со ссылками и индексами в базе данных MongoDB.

Оборудование: компьютерный класс.

Программное обеспечение: СУБД MongoDB 8.2.

2 CRUD-ОПЕРАЦИИ В СУБД MONGODB. ВСТАВКА ДАННЫХ. ВЫБОРКА ДАННЫХ

2.1 ВСТАВКА ДОКУМЕНТОВ В КОЛЛЕКЦИЮ

Практическое задание 2.1.1:

1. Создаем базу данных learn, заполняем коллекцию unicorns и отдельно добавляем еще одного единорога:

```
learn> use learn
[already on db learn]
learn> db.unicorns.insert({name: 'Horny', loves: ['carrot', 'papaya'], weight: 600, gender: 'm', vampires: 63});
[ db.unicorns.insert({name: 'Aurora', loves: ['carrot', 'grape'], weight: 450, gender: 'f', vampires: 43});
[ db.unicorns.insert({name: 'Unicrom', loves: ['energon', 'redbull'], weight: 984, gender: 'm', vampires: 182});
[ db.unicorns.insert({name: 'Rooooooodles', loves: ['apple'], weight: 575, gender: 'm', vampires: 99});
[ db.unicorns.insert({name: 'Solnara', loves: ['apple', 'carrot', 'chocolate'], weight: 550, gender: 'f', vampires: 80});
[ db.unicorns.insert({name: 'Ayna', loves: ['strawberry', 'lemon'], weight: 733, gender: 'f', vampires: 40});
[ db.unicorns.insert({name: 'Kenny', loves: ['grape', 'lemon'], weight: 690, gender: 'm', vampires: 39});
[ db.unicorns.insert({name: 'Raleigh', loves: ['apple', 'sugar'], weight: 421, gender: 'm', vampires: 2});
[ db.unicorns.insert({name: 'Leia', loves: ['apple', 'watermelon'], weight: 601, gender: 'f', vampires: 33});
[ db.unicorns.insert({name: 'Pilot', loves: ['apple', 'watermelon'], weight: 650, gender: 'm', vampires: 54});
[ db.unicorns.insert({name: 'Nimue', loves: ['grape', 'carrot'], weight: 540, gender: 'f'});
[
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a29bd27828a5697935f5421') }
}
]
learn> document = {name: 'Dunx', loves: ['grape', 'watermelon'], weight: 704, gender: 'm', vampires: 165}
[ db.unicorns.insert(document)
[
{
  acknowledged: true,
  insertedIds: { '0': ObjectId('6a29bd5d828a5697935f5422') }
}
]
```

2. Проверяем:

```
learn> db.unicorns.find()
[
{
  _id: ObjectId('6a29bd26828a5697935f5417'),
  name: 'Horny',
  loves: [ 'carrot', 'papaya' ],
  weight: 600,
  gender: 'm',
  vampires: 63
},
{
  _id: ObjectId('6a29bd27828a5697935f5418'),
  name: 'Aurora',
  loves: [ 'carrot', 'grape' ],
  weight: 450,
  gender: 'f',
  vampires: 43
},
{
  _id: ObjectId('6a29bd27828a5697935f5419'),
  name: 'Unicrom',
  loves: [ 'energon', 'redbull' ],
  weight: 984,
  gender: 'm',
  vampires: 182
},
{
  _id: ObjectId('6a29bd27828a5697935f541a'),
  name: 'Rooooooodles',
  loves: [ 'apple' ],
  weight: 575,
  gender: 'm',
  vampires: 99
},
]
```

```

    _id: ObjectId('6a29bd27828a5697935f541b'),
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    _id: ObjectId('6a29bd27828a5697935f541c'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  },
  {
    _id: ObjectId('6a29bd27828a5697935f541d'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  },
  {
    _id: ObjectId('6a29bd27828a5697935f541e'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  },
  {
    _id: ObjectId('6a29bd27828a5697935f541f'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    _id: ObjectId('6a29bd27828a5697935f5420'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 54
  },
  {
    _id: ObjectId('6a29bd27828a5697935f5421'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  },
  {
    _id: ObjectId('6a29bd5d828a5697935f5422'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  }
}

```

2.2 ВЫБОРКА ДАННЫХ ИЗ БД

Практическое задание 2.2.1:

1. Список самцов с сортировкой по имени:

```
learn> db.unicorns.find({gender: 'm'}).sort({name: 1})
[
  {
    _id: ObjectId('6a29bd5d828a5697935f5422'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  },
  {
    _id: ObjectId('6a29bd26828a5697935f5417'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 63
  },
  {
    _id: ObjectId('6a29bd27828a5697935f541d'),
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  },
  {
    _id: ObjectId('6a29bd27828a5697935f5420'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 54
  },
  {
    _id: ObjectId('6a29bd27828a5697935f541e'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  },
  {
    _id: ObjectId('6a29bd27828a5697935f541a'),
    name: 'Rooooooodles',
    loves: [ 'apple' ],
    weight: 575,
    gender: 'm',
    vampires: 99
  },
  {
    _id: ObjectId('6a29bd27828a5697935f5419'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 182
  }
]
```

2. Список первых трех самок с сортировкой по имени:

```
learn> db.unicorns.find({gender: 'f'}).sort({name: 1}).limit(3)
[
  {
    _id: ObjectId('6a29bd27828a5697935f5418'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  },
  {
    _id: ObjectId('6a29bd27828a5697935f541c'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 733,
    gender: 'f',
    vampires: 40
  },
  {
    _id: ObjectId('6a29bd27828a5697935f541f'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  }
]
```

3. Находим одну самку, которая любит морковь, двумя способами:


```
learn> db.unicorns.findOne({gender: 'f', loves: 'carrot'})
[
  {
    _id: ObjectId('6a29bd27828a5697935f5418'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  }
]
learn> db.unicorns.find({gender: 'f', loves: 'carrot'}).limit(1)
[
  {
    _id: ObjectId('6a29bd27828a5697935f5418'),
    name: 'Aurora',
    loves: [ 'carrot', 'grape' ],
    weight: 450,
    gender: 'f',
    vampires: 43
  }
]
```

Практическое задание 2.2.2:

1. Список самцов без информации о предпочтениях и поле:

```
learn> db.unicorns.find({gender: 'm'}, {loves: 0, gender: 0, _id: 0})
[
  { name: 'Horny', weight: 600, vampires: 63 },
  { name: 'Unicrom', weight: 984, vampires: 182 },
  { name: 'Roooooodles', weight: 575, vampires: 99 },
  { name: 'Kenny', weight: 690, vampires: 39 },
  { name: 'Raleigh', weight: 421, vampires: 2 },
  { name: 'Pilot', weight: 650, vampires: 54 },
  { name: 'Dunx', weight: 704, vampires: 165 }
]
```

Практическое задание 2.2.3:

1. Список единорогов в обратном порядке добавления:

```
learn> db.unicorns.find().sort({$natural: -1})
[
  {
    _id: ObjectId('6a29bd5d828a5697935f5422'),
    name: 'Dunx',
    loves: [ 'grape', 'watermelon' ],
    weight: 704,
    gender: 'm',
    vampires: 165
  },
  {
    _id: ObjectId('6a29bd27828a5697935f5421'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  },
  {
    _id: ObjectId('6a29bd27828a5697935f5420'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon' ],
    weight: 650,
    gender: 'm',
    vampires: 54
  },
  {
    _id: ObjectId('6a29bd27828a5697935f541f'),
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    _id: ObjectId('6a29bd27828a5697935f541e'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  },
]
```

```

{
  _id: ObjectId('6a29bd27828a5697935f541d'),
  name: 'Kenny',
  loves: [ 'grape', 'lemon' ],
  weight: 690,
  gender: 'm',
  vampires: 39
},
{
  _id: ObjectId('6a29bd27828a5697935f541c'),
  name: 'Ayna',
  loves: [ 'strawberry', 'lemon' ],
  weight: 733,
  gender: 'f',
  vampires: 40
},
{
  _id: ObjectId('6a29bd27828a5697935f541b'),
  name: 'Solnara',
  loves: [ 'apple', 'carrot', 'chocolate' ],
  weight: 550,
  gender: 'f',
  vampires: 80
},
{
  _id: ObjectId('6a29bd27828a5697935f541a'),
  name: 'Rooooooooodles',
  loves: [ 'apple' ],
  weight: 575,
  gender: 'm',
  vampires: 99
},
{
  _id: ObjectId('6a29bd27828a5697935f5419'),
  name: 'Unicrom',
  loves: [ 'energon', 'redbull' ],
  weight: 984,
  gender: 'm',
  vampires: 182
},
{
  _id: ObjectId('6a29bd27828a5697935f5418'),
  name: 'Aurora',
  loves: [ 'carrot', 'grape' ],
  weight: 450,
  gender: 'f',
  vampires: 43
},
{
  _id: ObjectId('6a29bd26828a5697935f5417'),
  name: 'Horny',
  loves: [ 'carrot', 'papaya' ],
  weight: 600,
  gender: 'm',
  vampires: 63
}
]

```

Практическое задание 2.2.4:

1. Список единорогов с названием первого любимого предпочтения без идентификатора:

```

learn> db.unicorns.find({}, {name: 1, loves: {$slice: 1}, _id: 0})
[
  { name: 'Horny', loves: [ 'carrot' ] },
  { name: 'Aurora', loves: [ 'carrot' ] },
  { name: 'Unicrom', loves: [ 'energon' ] },
  { name: 'Rooooooooodles', loves: [ 'apple' ] },
  { name: 'Solnara', loves: [ 'apple' ] },
  { name: 'Ayna', loves: [ 'strawberry' ] },
  { name: 'Kenny', loves: [ 'grape' ] },
  { name: 'Raleigh', loves: [ 'apple' ] },
  { name: 'Leia', loves: [ 'apple' ] },
  { name: 'Pilot', loves: [ 'apple' ] },
  { name: 'Nimue', loves: [ 'grape' ] },
  { name: 'Dunx', loves: [ 'grape' ] }
]

```

2.3 ЛОГИЧЕСКИЕ ОПЕРАТОРЫ

Практическое задание 2.3.1:

1. Список самок единорогов весом от 500 кг до 700 кг без идентификатора:

```
learn> db.unicorns.find({gender: 'f', weight: {$gte: 500, $lte: 700}}, {_id: 0})
[
  {
    name: 'Solnara',
    loves: [ 'apple', 'carrot', 'chocolate' ],
    weight: 550,
    gender: 'f',
    vampires: 80
  },
  {
    name: 'Leia',
    loves: [ 'apple', 'watermelon' ],
    weight: 601,
    gender: 'f',
    vampires: 33
  },
  {
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
```

Практическое задание 2.3.2:

1. Список самцов единорогов весом от 500 кг и предпочитающих grape и lemon без идентификатора:

```
learn> db.unicorns.find({gender: 'm', weight: {$gte: 500}, loves: {$all: ['grape', 'lemon']}}, {_id: 0})
[
  {
    name: 'Kenny',
    loves: [ 'grape', 'lemon' ],
    weight: 690,
    gender: 'm',
    vampires: 39
  }
]
```

Практическое задание 2.3.3:

1. Список всех единорогов, которые не имеют ключ vampires:

```
learn> db.unicorns.find({vampires: {$exists: false}})
[
  {
    _id: ObjectId('6a29bd27828a5697935f5421'),
    name: 'Nimue',
    loves: [ 'grape', 'carrot' ],
    weight: 540,
    gender: 'f'
  }
]
```

Практическое задание 2.3.4:

1. Список имен самцов единорогов с информацией об их первом предпочтении:

```
learn> db.unicorns.find({gender: 'm'}, {name: 1, loves: {$slice: 1}, _id: 0}).sort({name: 1})
[
  { name: 'Dunx', loves: [ 'grape' ] },
  { name: 'Horny', loves: [ 'carrot' ] },
  { name: 'Kenny', loves: [ 'grape' ] },
  { name: 'Pilot', loves: [ 'apple' ] },
  { name: 'Raleigh', loves: [ 'apple' ] },
  { name: 'Rooooooodles', loves: [ 'apple' ] },
  { name: 'Unicrom', loves: [ 'energon' ] }
]
```

3 ЗАПРОСЫ К БАЗЕ ДАННЫХ MONGODB.

ВЫБОРКА ДАННЫХ. ВЛОЖЕННЫЕ ОБЪЕКТЫ. ИСПОЛЬЗОВАНИЕ КУРСОРОВ. АГРЕГИРОВАННЫЕ ЗАПРОСЫ. ИЗМЕНЕНИЕ ДАННЫХ

3.1 ЗАПРОС К ВЛОЖЕННЫМ ОБЪЕКТАМ

Практическое задание 3.1.1:

1. Создаем коллекцию towns:

```
learn> db.towns.insert({name:"Punxsutawney", population: 6200, last_census: ISODate("2008-01-31"), famous_for:["phil the groundhog"], mayor:{name:"Jim Wehrle"}});
|
| db.towns.insert({name:"New York", population: 22200000, last_census: ISODate("2009-07-31"), famous_for:["status of liberty","food"], mayor:{name:"Michael Bloomberg", party:"I"}});
|
| db.towns.insert({name:"Portland", population: 528000, last_census: ISODate("2009-07-20"), famous_for:["beer","food"], mayor:{name:"Sam Adams", party:"D"}});
|
| {
|   acknowledged: true,
|   insertedIds: { '0': ObjectId('6a29c9d6828a5697935f5425') }
| }
|
```

2. Список городов с независимыми мэрами с выводом названия города и информации о мэре:

```
learn> db.towns.find({"mayor.party": "I"}, {name: 1, mayor: 1, _id: 0})
[
  { name: 'New York', mayor: { name: 'Michael Bloomberg', party: 'I' } }
]
```

3. Список беспартийных мэров с выводом названия города и информации о мэре:

```
learn> db.towns.find({"mayor.party": {$exists: false}}, {name: 1, mayor: 1, _id: 0})
[ { name: 'Punxsutawney', mayor: { name: 'Jim Wehrle' } } ]
```

Практическое задание 3.1.2:

1. Функция для вывода самцов, курсор для данного списка из первых двух особей с сортировкой по алфавиту и вывод результата с использованием forEach:

```
learn> fn_male = function(doc){ return doc.gender == 'm'; }
[Function: fn_male]
learn> var cursor = db.unicorns.find({gender: 'm'}).sort({name: 1}).limit(2); null;
null
learn> cursor.forEach(function(obj){
|   print(obj.name);
| })
Dunx
Horny
```

3.2 АГРЕГИРОВАННЫЕ ЗАПРОСЫ

Практическое задание 3.2.1:

1. Вывод количества самок единорогов весом от 500 кг до 600 кг:

```
learn> db.unicorns.find({gender: 'f', weight: {$gte: 500, $lte: 600}}).count()
(node:33680) [MONGODB DRIVER] Warning: cursor.count is deprecated and will be removed in the next major version, please use
`collection.estimatedDocumentCount` or `collection.countDocuments` instead
(Use `node --trace-warnings ...` to show where the warning was created)
2
```

Практическое задание 3.2.2:

1. Вывод списка предпочтений:

```
learn> db.unicorns.distinct("loves")
[
  'apple',      'carrot',
  'chocolate', 'energon',
  'grape',      'lemon',
  'papaya',     'redbull',
  'strawberry', 'sugar',
  'watermelon'
]
```

Практическое задание 3.2.3:

1. Подсчет количества особей обоих полов:

```
learn> db.unicorns.aggregate({"$group": {_id: "$gender", count: {$sum: 1}}})
[ { _id: 'm', count: 7 }, { _id: 'f', count: 5 } ]
```

3.3 РЕДАКТИРОВАНИЕ ДАННЫХ

Практическое задание 3.3.1:

1. Добавляем нового единорога и проверяем, добавился ли он (save() устарела, поэтому я использовала insertOne()):

```
[learn> db.unicorns.save({name:'Barney', loves:['grape'], weight: 340, gender:'m'})
TypeError: db.unicorns.save is not a function
[learn> db.unicorns.insertOne({name:'Barney', loves:['grape'], weight: 340, gender:'m'})
{
  acknowledged: true,
  insertedId: ObjectId('6a29d482828a5697935f5426')
}
[learn> db.unicorns.find({name: 'Barney'})
[
  {
    _id: ObjectId('6a29d482828a5697935f5426'),
    name: 'Barney',
    loves: [ 'grape' ],
    weight: 340,
    gender: 'm'
  }
]
```

Практическое задание 3.3.2:

1. Меняем вес единорога Айна и количество ее вампиров, проверяем:

```
[learn> db.unicorns.updateOne(
|   {name: "Ayna"},
|   {$set: {weight: 800, vampires: 51}}
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
[learn> db.unicorns.find({name: 'Ayna'})
[
  {
    _id: ObjectId('6a29bd27828a5697935f541c'),
    name: 'Ayna',
    loves: [ 'strawberry', 'lemon' ],
    weight: 800,
    gender: 'f',
    vampires: 51
  }
]
```

Практическое задание 3.3.3:

1. Меняем предпочтения единорога Raleigh, проверяем:

```
[learn> db.unicorns.updateOne(
|   {name: 'Raleigh'},
|   {$set: {loves: ['apple', 'sugar', 'redbull']}}
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
[learn> db.unicorns.find({name: 'Raleigh'})
[
  {
    _id: ObjectId('6a29bd27828a5697935f541e'),
    name: 'Raleigh',
    loves: [ 'apple', 'sugar', 'redbull' ],
    weight: 421,
    gender: 'm',
    vampires: 2
  }
]
```

Практическое задание 3.3.4:

1. Увеличиваем количество вампиров всех единорогов на 5, проверяем содержимое коллекции:

```
learn> db.unicorns.updateMany(
|   {gender: 'm'},
|   {$inc: {vampires: 5}}
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 8,
  modifiedCount: 8,
  upsertedCount: 0
}
learn> db.unicorns.find({gender: 'm'}, {name: 1, vampires: 1, _id: 0})
[
  { name: 'Horny', vampires: 68 },
  { name: 'Unicrom', vampires: 187 },
  { name: 'Rooooooodles', vampires: 104 },
  { name: 'Kenny', vampires: 44 },
  { name: 'Raleigh', vampires: 7 },
  { name: 'Pilot', vampires: 59 },
  { name: 'Dunx', vampires: 170 },
  { name: 'Barney', vampires: 5 }
]
```

Практическое задание 3.3.5:

1. Изменяем информацию о городе Portland (мэр этого города теперь беспартийный), проверяем:

```
learn> db.towns.updateOne(
|   {name: "Portland"},
|   {$unset: {"mayor.party": 1}}
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.towns.find({name: "Portland"})
[
  {
    _id: ObjectId('6a29c9d6828a5697935f5425'),
    name: 'Portland',
    population: 528000,
    last_census: ISODate('2009-07-20T00:00:00.000Z'),
    famous_for: [ 'beer', 'food' ],
    mayor: { name: 'Sam Adams' }
  }
]
```

Практическое задание 3.3.6:

1. Изменяем информацию о самце единорога Pilot (теперь он любит и шоколад), проверяем:

```
learn> db.unicorns.updateOne(
|   {name: 'Pilot'},
|   {$push: {loves: 'chocolate'}}
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.find({name: 'Pilot'})
[
  {
    _id: ObjectId('6a29bd27828a5697935f5420'),
    name: 'Pilot',
    loves: [ 'apple', 'watermelon', 'chocolate' ],
    weight: 650,
    gender: 'm',
    vampires: 59
  }
]
```

Практическое задание 3.3.7:

1. Изменяем информацию о самке единорога Auroga (теперь она любит еще и сахар, и лимоны), проверяем:

```
learn> db.unicorns.updateOne(
|   {name: 'Aurora'},
|   {$addToSet: {loves: {$each: ['sugar', 'lemon']}}}
| )
|
| {
|   acknowledged: true,
|   insertedId: null,
|   matchedCount: 1,
|   modifiedCount: 1,
|   upsertedCount: 0
| }
learn> db.unicorns.find({name: 'Aurora'})
|
| {
|   _id: ObjectId('6a29bd27828a5697935f5418'),
|   name: 'Aurora',
|   loves: [ 'carrot', 'grape', 'sugar', 'lemon' ],
|   weight: 450,
|   gender: 'f',
|   vampires: 43
| }
|
```

3.4 УДАЛЕНИЕ ДАННЫХ ИЗ КОЛЛЕКЦИИ

Практическое задание 3.4.1:

1. Удаляем документы с беспартийными мэрами:

```
learn> db.towns.deleteMany({"mayor.party": {$exists: false}})
{ acknowledged: true, deletedCount: 2 }
```

2. Проверяем:

```
learn> db.towns.find()
|
| {
|   _id: ObjectId('6a29c9d6828a5697935f5424'),
|   name: 'New York',
|   population: 22200000,
|   last_census: ISODate('2009-07-31T00:00:00.000Z'),
|   famous_for: [ 'status of liberty', 'food' ],
|   mayor: { name: 'Michael Bloomberg', party: 'I' }
| }
|
```

3. Удаляем все:

```
learn> db.towns.deleteMany({})
{ acknowledged: true, deletedCount: 1 }
```

4. Проверяем:

```
learn> show collections
towns
unicorns
learn> db.towns.find()
```

4 ССЫЛКИ И РАБОТА С ИНДЕКСАМИ В БАЗЕ ДАННЫХ MONGODB

4.1 ССЫЛКИ В БД

Практическое задание 4.1.1:

1. Создаем документы зон обитания и проверяем коллекцию:

```
learn> db.habitats.insertOne({_id: "everfree", name: "Everfree Forest", description: "Mysterious forest full of magic and wild creatures"})
| db.habitats.insertOne({_id: "crystal", name: "Crystal Mountains", description: "High mountains with crystal caves and pure air"})
| db.habitats.insertOne({_id: "starlight", name: "Starlight Meadows", description: "Endless meadows under the starry sky"})
{ acknowledged: true, insertedId: 'starlight' }
learn> db.habitats.find()
[
  {
    _id: 'everfree',
    name: 'Everfree Forest',
    description: 'Mysterious forest full of magic and wild creatures'
  },
  {
    _id: 'crystal',
    name: 'Crystal Mountains',
    description: 'High mountains with crystal caves and pure air'
  },
  {
    _id: 'starlight',
    name: 'Starlight Meadows',
    description: 'Endless meadows under the starry sky'
  }
]
```

2. Привязываем Horny к Everfree Forest, а Unicrom к Crystal Mountains:

```
learn> db.unicorns.updateOne(
|   {name: "Horny"},
|   {$set: {habitat: {$ref: "habitats", $id: "everfree"}}}
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
learn> db.unicorns.updateOne(
|   {name: "Unicrom"},
|   {$set: {habitat: {$ref: "habitats", $id: "crystal"}}}
| )
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

3. Проверяем:

```
learn> db.unicorns.find({name: "Horny"})
[
  {
    _id: ObjectId('6a29bd26828a5697935f5417'),
    name: 'Horny',
    loves: [ 'carrot', 'papaya' ],
    weight: 600,
    gender: 'm',
    vampires: 68,
    habitat: DBRef('habitats', 'everfree')
  }
]
learn> db.unicorns.find({name: "Unicrom"})
[
  {
    _id: ObjectId('6a29bd27828a5697935f5419'),
    name: 'Unicrom',
    loves: [ 'energon', 'redbull' ],
    weight: 984,
    gender: 'm',
    vampires: 187,
    habitat: DBRef('habitats', 'crystal')
  }
]
learn> db.unicorns.find({ habitat: { $exists: true } }, { name: 1, habitat: 1, _id: 0 })
[
  { name: 'Horny', habitat: DBRef('habitats', 'everfree') },
  { name: 'Unicrom', habitat: DBRef('habitats', 'crystal') }
]
```

4.2 НАСТРОЙКА ИНДЕКСОВ

Практическое задание 4.2.1:

1. Проверяем есть ли дубликаты имен:

```
learn> db.unicorns.createIndex({name: 1}, {unique: true})
name_1
```

Практическое задание 4.3.1:

2. Проверяем все индексы коллекции unicorns:

```
learn> db.unicorns.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_', },
  { v: 2, key: { name: 1 }, name: 'name_1', unique: true }
]
```

3. Удаляем все индексы, кроме индекса для идентификатора, затем пытаемся удалить этот индекс:

```
learn> db.unicorns.dropIndex("name_1")
{ nIndexesWas: 2, ok: 1 }
learn> db.unicorns.dropIndex("_id_")
MongoServerError[InvalidOptions]: cannot drop _id index
```

4.4 ПЛАН ЗАПРОСА

Практическое задание 4.4.1:

1. Заполняем коллекцию числами от 0 до 99999:

```
learn> for(let i = 0; i < 100000; i++) {
|   db.numbers.insertOne({value: i})
| }
{
  acknowledged: true,
  insertedId: ObjectId('6a29e74c828a56979360dac6')
}
```

2. Выбираем последние 4 документа:

```
learn> db.numbers.find().sort({value: -1}).limit(4)
[
  { _id: ObjectId('6a29e74c828a56979360dac6'), value: 99999 },
  { _id: ObjectId('6a29e74c828a56979360dac5'), value: 99998 },
  { _id: ObjectId('6a29e74c828a56979360dac4'), value: 99997 },
  { _id: ObjectId('6a29e74c828a56979360dac3'), value: 99996 }
]
```

3. Анализируем план выполнения запроса 2, сколько времени потребовалось:

```
executionStats: {
  executionSuccess: true,
  nReturned: 4,
  executionTimeMillis: 66,
  totalKeysExamined: 0,
  totalDocsExamined: 100000,
  executionStages: {
```

4. Создаем индекс для ключа value, проверяем все индексы коллекции numbers:

```
learn> db.numbers.createIndex({value: 1})
value_1
learn> db.numbers.getIndexes()
[
  { v: 2, key: { _id: 1 }, name: '_id_', },
  { v: 2, key: { value: 1 }, name: 'value_1' }
]
```

5. Выполняем запрос 2 повторно:

```
learn> db.numbers.find().sort({value: -1}).limit(4)
[
  { _id: ObjectId('6a29e74c828a56979360dac6'), value: 99999 },
  { _id: ObjectId('6a29e74c828a56979360dac5'), value: 99998 },
  { _id: ObjectId('6a29e74c828a56979360dac4'), value: 99997 },
  { _id: ObjectId('6a29e74c828a56979360dac3'), value: 99996 }
]
```

6. Снова анализируем план выполнения запроса 2, сколько времени потребовалось:

```
executionStats: {
  executionSuccess: true,
  nReturned: 4,
  executionTimeMillis: 0,
  totalKeysExamined: 4,
  totalDocsExamined: 4,
  executionStages: {
```

7. Время выполнения запроса без индекса составило 66 мс, а с индексом 1 мс. Запрос с индексом оказался эффективнее, так как без индекса MongoDB сканирует всю коллекцию и сортирует документы, а с индексом использует направленный обход индекса, что выходит гораздо быстрее.

Вывод: В ходе выполнения лабораторной работы были приобретены практические навыки работы с документо-ориентированной СУБД MongoDB в консоли mongosh. Были изучены принципы создания баз данных и коллекций, а также реализованы основные CRUD-операции: вставка, выборка, обновление и удаление документов. Освоена работа со сложными структурами данных, включая вложенные объекты и массивы, применение логических операторов, курсоров для обработки выборок и агрегированных запросов для группировки данных. Кроме того, были рассмотрены способы связывания коллекций через ссылки и создание индексов. На практике проверено значительное влияние индексов на скорость выполнения запросов, что позволило убедиться в их эффективности по сравнению с полным перебором коллекции.